

MelodyGenome: 基于深度学习与遗传算法的 自动机器作曲项目

小组成员:

宋澄 唐天睿 姚佳恒 韦延睿 李孟桐 刘一凡
(《音乐与数学》课程选题 3 第 1 小组)

摘要

本文提出了 *MelodyGenome*, 这是一个结合了遗传算法与深度学习的单声部旋律生成框架。我们首先构建了一个包含多种风格的旋律的数据集, 在生成算法方面, 我们实现了包含移调、逆行、倒影等乐理变换算子的遗传算法。为解决传统遗传算法中适应度函数设计主观性强的问题, 本文设计并训练了四种不同架构的神经网络 (ClassicalResNet, ACGAttentionNet, PopTransformer, CombinedHybridNet) 作为评估器。实验结果表明, 混合了显式乐理规则与神经网络评分的适应度函数有效引导了种群进化, 生成兼具结构性与音乐性的旋律片段。项目在 github 开源<https://github.com/Stemiaya/MelodyGenome>

关键词: 机器作曲; 遗传算法; 深度学习; 适应度函数

项目贡献分工:

姚佳恒、刘一凡: 数据爬取、数据预处理、标签生成

唐天睿: 遗传算法逻辑 (`main.py`)、Rule-based 适应度函数、github 仓库维护

宋澄: ai 模型架构设计、模型训练与优化调整、报告撰写与整合

韦延睿: 模型实验部分与可视化乐谱分析

李孟桐: 项目整体架构

1 引言

机器作曲旨在利用形式化的方法生成音乐结构。遗传算法 (Genetic Algorithm) 作为一种启发式搜索算法, 适合在巨大的音乐组合空间中寻找最优解。GA 在音乐生成中的关键是**适应度函数**——即如何定义一个函数来衡量一段旋律是否动听。传统的机器作曲方法主要依赖于马尔可夫链或基于规则的语法, 这些方法虽然能保证乐理上的正确性, 但往往需要考虑较繁杂的乐理因素, 也缺乏一定的创新性。

针对上述问题, 我们小组提出了 *MelodyGenome*。我们的主要贡献如下:

1. **数据集构建与量化:** 从 Muscore 收集数据, 并基于用户交互数据构建了客观的旋律评分标签。

2. **融合乐理的遗传算子**：我们考虑了多种旋律变换，引入了如交叉、变异，以及逆行、倒影等具有音乐学意义的变换算子。
3. **神经适应度评估器**：训练了多种深度神经网络架构作为遗传算法的适应度函数。

2 数据获取与预处理

为了训练神经网络，我们需要构建一个带有分数标签的旋律数据集。

2.1 数据来源

项目数据来源于 *musescore.com* 网站。我们将爬取到的乐谱经过切片和筛选后得到训练样本。数据根据风格被划分为三类：

- **ACG & OST** (53 条)：动漫游戏原声，特征是旋律线清晰，包含大量 16 分音符，节奏感强。在网站上体现为 anime, game, ost, soundtrack, movie, film, theme, opening, ending, ghibli, touhou 等关键词。
- **Classical & Instrumental** (69 条)：古典与纯器乐，强调结构严谨与复调技巧。在网站上体现为 classical, piano, orchestra, baroque, romantic, sonata, etude, chopin, bach, beethoven 等关键词。
- **Pop & Contemporary** (920 条)：流行与现代音乐，具有标准化的和弦走向和段落结构。在网站上体现为 pop, rock, electronic, jazz, ballad, song, vocal, dance, 及所有未标记风格的数据。

Explore classical sheet music

Merry-Go-Round Of Life
Howl's Moving Castle
Piano Tutorial
PianoChannel
1 part • 7 pages • 05:15 • Jul 5, 2017 • 7.3M views • 250.1K saves
★★★★★ 58.2K votes
Solo Piano
Piano

Intermediate

River Flows In You
emmy langevin PRO
1 part • 4 pages • 02:47 • Jan 25, 2017 • 8.4M views • 254K saves
★★★★★ 54.6K votes
Solo Piano
Piano

Intermediate

Canon in D
Pachelbel
lemontart PRO
1 part • 6 pages • 04:05 • Jul 2, 2015 • 5.2M views • 170.2K saves
★★★★★ 42.1K votes
Solo Piano
Piano

Intermediate

Für Elise in A Minor
Beethoven
ClassicMan PRO
1 part • 3 pages • 02:37 • Dec 22, 2011 • 4.6M views • 148K saves
★★★★★ 36.7K votes
Solo Piano
Piano

Intermediate

Clair de Lune
Debussy
ClassicMan PRO
1 part • 6 pages • 04:36 • Aug 6, 2012 • 5.1M views • 155.7K saves
★★★★★ 31.3K votes
Solo Piano
Piano

Advanced

Nocturne Op 9 No.2
Chopin
Rodila
1 part • 4 pages • 03:40 • Aug 14, 2017 • 3.8M views • 135.5K saves
★★★★★ 24.4K votes
Solo Piano
Piano

Advanced

图 1: musescore.com 网站数据来源实例

2.2 数据预处理与编码

我们对原始 MIDI/MusicXML 数据进行了如下预处理：

1. **筛选与切片**：仅保留 4/4 拍的乐谱，从中随机截取 4 小节作为单条数据。
2. **相对音高编码**：我们将所有音符作为音类处理，映射到当前调式的相对音级上。
 - 1 ~ 12：代表一个八度内的相对音高（1 为主音）。
 - 0：代表休止符。
 - 13：代表延音（即延续前一个音的时值）。
3. **向量化**：以 16 分音符为最小时间单位（0.25 拍）。4 小节共包含 $4 \times 4 \times 4 = 64$ 个时间步。因此，每条旋律被表示为一个长度为 64 的整数向量。

2.3 标签生成

我们数据集的一个核心创新是基于 Musescore 社区反馈生成真实标签。我们将元数据（浏览量、点赞数、评论数、评分）加权成 $\text{mid_score}(S)$ 。原始数据来源于 Musescore 每一份乐谱的浏览量 (v)、点赞数 (l)、评论数 (c)、评分 (r) 及打分人数 (n)。

首先，我们根据数据分布定义了以下归一化常数以防止极端值影响：

- 最大浏览量 $V_{max} = 1,000,000$
- 最多点赞数 $L_{max} = 100,000$
- 最多评论数 $C_{max} = 1,000$
- 最高评分 $R_{max} = 5.0$
- 打分活跃度权重 $K = 10$

考虑到浏览量和点赞数通常呈现长尾分布，我们对这两项采用了对数归一化；对评论和活跃度采用了截断归一化。各分项得分计算公式及权重加和公式如下：

$$S_{view} = \frac{\log(v+1)}{\log(V_{max}+1)} \quad (1)$$

$$S_{like} = \frac{\log(l+1)}{\log(L_{max}+1)} \quad (2)$$

$$S_{comment} = \min\left(1.0, \frac{c}{C_{max}}\right) \quad (3)$$

$$S_{rating} = \frac{r}{R_{max}} \quad (4)$$

$$S_{count} = \min\left(1.0, \frac{n}{v} \times K\right) \quad (5)$$

$$S_{\text{mid}} = 0.5 \cdot S_{\text{view}} + 0.3 \cdot S_{\text{like}} + 0.05 \cdot S_{\text{comment}} + 0.1 \cdot S_{\text{rating}} + 0.05 \cdot S_{\text{count}} + \epsilon \quad (6)$$

其中，噪声项 $\epsilon \sim U(-5, 5)$ 。

随后，我们根据当前数据集分数标签的平均值与标准差，以 rule-based fitness 函数（后文提到）对数据集的打分为基准，对 mid_score 的分布进行调整，得到最终的 final_score ：

$$S_{\text{final}} = \frac{S_{\text{mid}} - \mu_{\text{mid}}}{\sigma_{\text{mid}}} \times \sigma_{\text{f}} + \mu_{\text{f}} \quad (7)$$

这使得 final_score 既满足数据集原始分数标签的分布，其数值也与 rule-based fitness 函数的评分相近，避免了使用混合评分时的数据误差。

2.4 数据实例解析

为了更直观地展示数据处理结果，我们从处理好的数据集中选取一条样本解析：

表 1: 处理后的数据样本示例	
字段	数值
score_id	11603002
key	A (A 大调)
segment_start	116 (起始于第 116 小节)
final_score	275.3397 (综合评分)

该样本对应的 **melody_vector** 如下所示：

[8, 13, 13, 13, 6, 13, 13, 13, 5, 13, 13, 13, ..., 1, 0, 10, ...]

- **8, 13, 13, 13**：数字 8 代表该调式下的相对音高（在 A 大调中对应 E 音）。随后的三个 13 表示该音符被延长了 3 个时间单位（共持续 1 拍，即四分音符）。

通过这种方式，原本复杂的五线谱信息被压缩为适合神经网络处理的定长数值向量，同时保留了音高、时值和休止符的节奏信息。

3 遗传算法与适应度函数

3.1 遗传算法设计

我们采用的遗传算法如下所示：

Algorithm 1 MelodyGenome 遗传进化主流程

Require: 种群大小 N , 迭代代数 G , 变异率 P_m

```
1: Initialization: 使用固定种子初始化种群  $P$  (Seeded Mode)。  
2: for  $gen = 1$  to  $G$  do  
3:   Evaluation: 计算种群  $P$  中所有个体的适应度。  
4:   Selection: 按适应度排序, 保留前 50% 个体作为精英集合  $E$ 。  
5:    $P_{next} \leftarrow \emptyset$   
6:   while  $|P_{next}| < N$  do  
7:      $p_1, p_2 \leftarrow \text{RandomSample}(E, 2)$  {从精英中随机选择父母}  
8:      $c_1, c_2 \leftarrow \text{Crossover}(p_1, p_2)$  {两点交叉}  
9:      $c_1 \leftarrow \text{Mutation}(c_1)$ ,  $c_2 \leftarrow \text{Mutation}(c_2)$  {变异}  
10:     $c_1 \leftarrow \text{MusicalTransform}(c_1)$  {应用乐理变换 (移调/逆行/倒影/逆行倒影)}  
11:     $P_{next}.append(c_1, c_2)$   
12:   end while  
13:    $P \leftarrow P_{next}[0 : N]$  {截取至种群大小}  
14:   Log: 输出当前代最佳适应度。  
15: end for  
16: Output: 返回最终种群中最优的 Top-k 个体。
```

3.2 适应度模型 Fitness Models

适应度函数决定了进化的方向。本项目实现了三种模式：**Rule-based**（基于规则）、**AI-based**（基于神经网络）和 **Hybrid**（Rule 和 AI 混合模式）。

3.2.1 Rule-based 架构设计原理

基于传统乐理, 我们设计了一套显式评分函数 F_{rule} 。该函数主要包含以下维度的约束: [1]

- **和声稳定性:** 算法检测强拍上的音符, 若其属于主和弦音 (如 I、III、V 级音), 则给予正向激励。此外, 为了保证乐段的终止感, 算法强制奖励结束音回归主音的旋律片段。
- **旋律平滑度:** 根据音程大小进行评分。函数奖励二度、三度的级进, 惩罚超过纯五度的大跳, 并对旋律方向的频繁无序改变施加惩罚。
- **结构与丰富度:** 在节奏上, 惩罚过多的碎片化切分音; 在音高上, 惩罚音符种类过少 (即单调重复) 的个体, 确保旋律一定的生动性。

3.2.2 AI 模型架构设计原理

音乐风格具有多样性, 不同流派的音乐在结构、旋律线和节奏型上表现出截然不同的特征。为了更精准地评估生成旋律的质量, 我们针对数据集的特性设计了四种差异化的神经网络架构:

1. ClassicalResNet (针对古典音乐):

- **设计动机**: 古典音乐通常具有严谨的层级结构和显著的局部动机。一个短小的音乐动机可能会在曲目中多次出现、模进或变奏。
- **架构特性**: 我们采用了基于一维卷积的残差网络。卷积核在时间轴上滑动，能够有效提取局部音型特征，具有平移不变性；而残差连接则允许网络构建更深的层级，捕捉古典音乐中复杂的结构化特征，避免梯度消失。

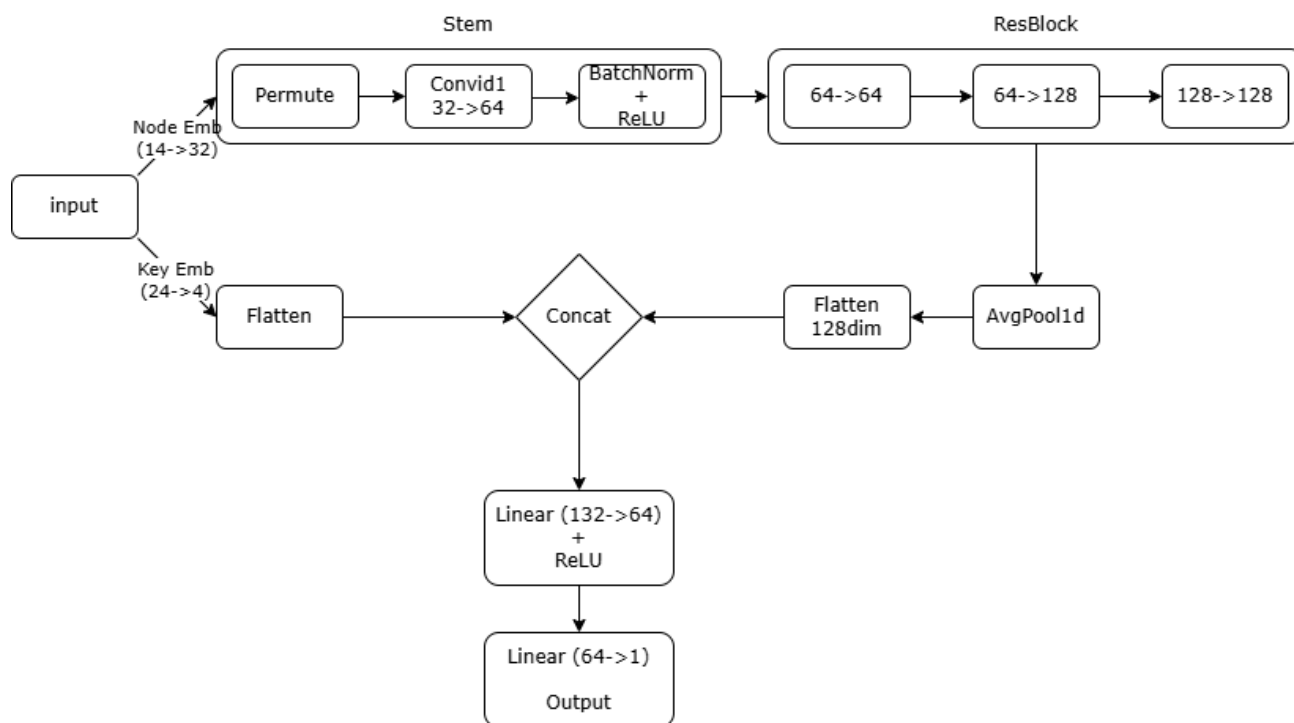


图 2: ClassicalResNet 神经网络架构示意图

2. ACGAttentionNet (针对 ACG 音乐):

- **设计动机**: ACG 音乐通常强调情感的起伏和叙事性，旋律线中往往存在特定的“高潮段落”，这些关键音符决定了旋律的感染力。
- **架构特性**: 采用双向 LSTM 提取时序上下文特征，确保模型能理解音符的前后关联。同时引入注意力机制，允许模型在评估时自动聚焦于旋律中最具表现力的关键片段，这与人类听众对 ACG 音乐的审美方式其实一致。

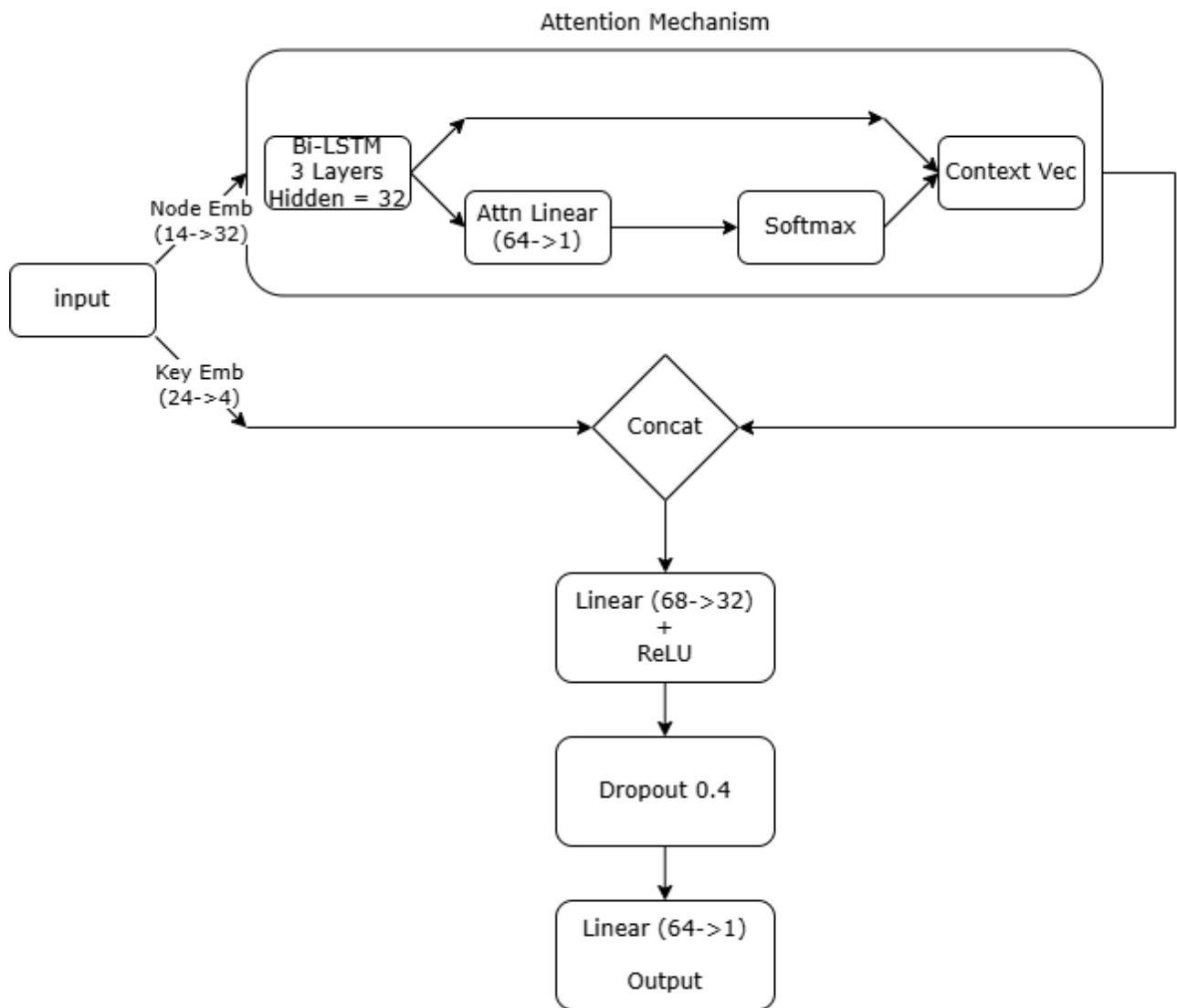


图 3: ACGAttentionNet 神经网络架构示意图

3. PopTransformer (针对流行音乐):

- **设计动机:** 流行音乐具有高度标准化的曲式结构，且往往存在重复与呼应。
- **架构特性:** 基于 Transformer Encoder，利用自注意力机制捕捉序列中的长距离依赖。与 RNN 不同，Transformer 可以同时看到整个序列，从而更好地理解流行音乐中跨小节、跨乐句的全局结构重复性。

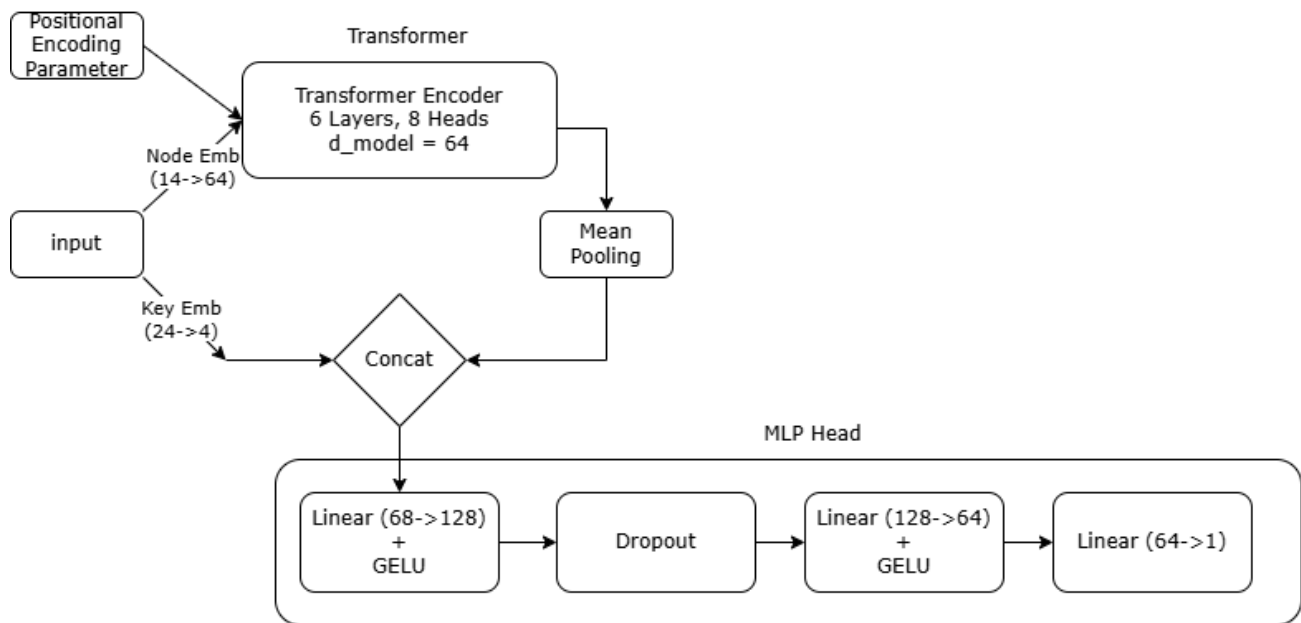


图 4: PopTransformer 神经网络架构示意图

4. CombinedHybridNet (通用融合模型):

- **设计动机**: 当面对混合风格或未标记风格的数据时, 单一视角的模型可能存在偏差。我们需要一个可以把握整体流向的鲁棒模型。
- **架构特性**: 采用了 Dual-stream 架构进行特征融合:
 - **Stream A (CNN)**: 负责提取局部的节奏模式和音程特征。
 - **Stream B (GRU)**: 负责捕捉旋律随时间演变的时序动态。

两路特征最后与调性嵌入拼接, 通过全连接层输出, 兼顾了微观与宏观视角。

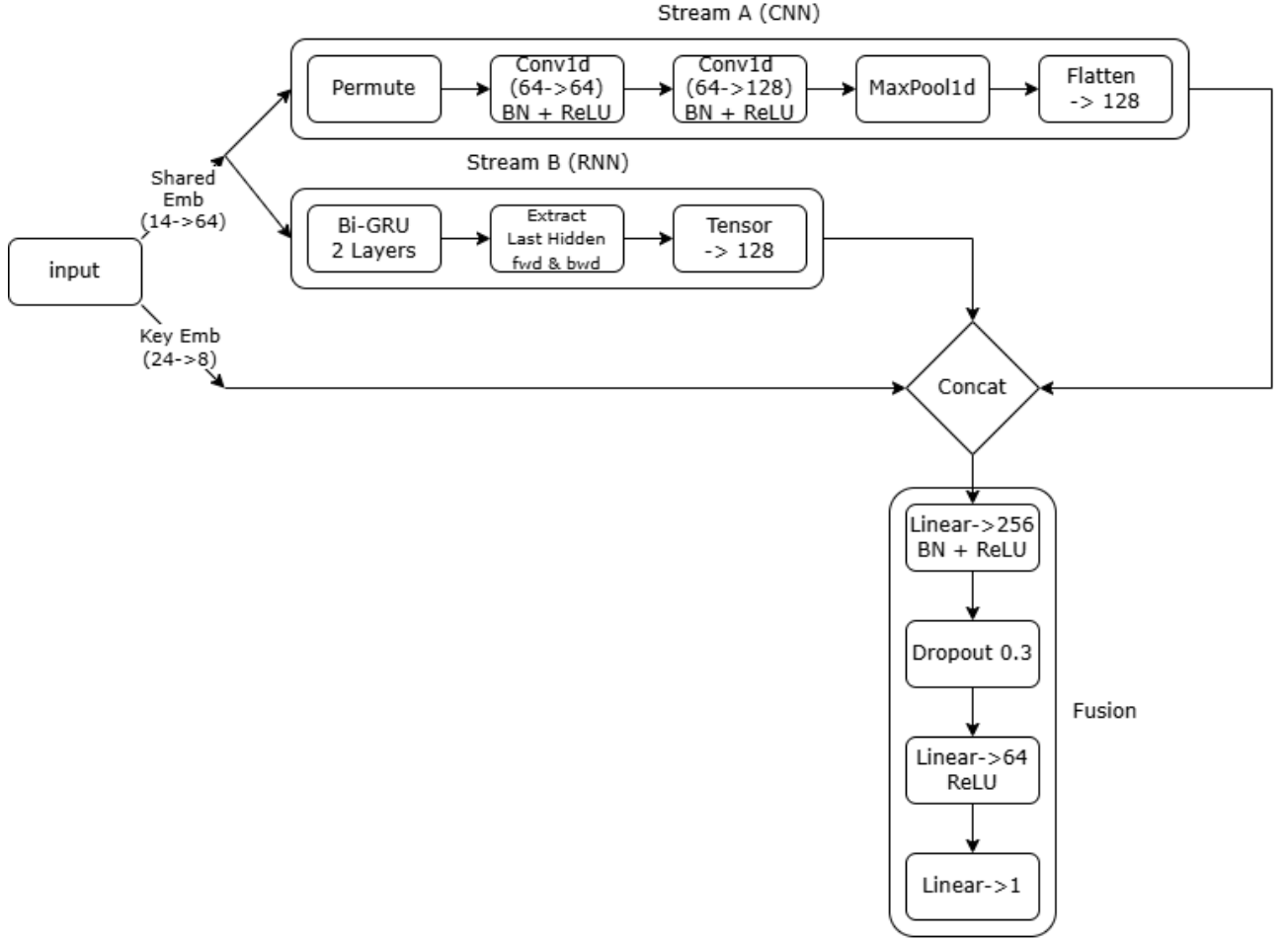


图 5: CombinedHybridNet 神经网络架构示意图

3.2.3 Hybrid 架构设计原理

单纯依赖规则往往会导致生成的旋律缺乏创造性；而单纯依赖 AI 模型则可能出现“幻觉”或乐理错误。为了平衡规范性与艺术性，我们提出了混合适应度函数。

该策略采用加权求和的方式融合两者的优势：

$$F_{total} = \alpha \cdot F_{rule} + (1 - \alpha) \cdot F_{AI} \quad (8)$$

其中，我们设置了规则评分 F_{rule} 的阈值机制：当个体的规则分低于特定阈值时，直接判定为不合格；仅当其满足基本乐理要求后，才更多地依赖 F_{AI} 来决定最终的分。在实验中，我们设置 $\alpha = 0.4$ 。

4 实验与分析

4.1 实验设置

4.1.1 初始种群配置

为了探究算法在不同输入条件下的收敛表现，我们构建了三类初始种群（各包含 100 个个体）：

- **真实旋律组**：从预处理后的 Musescore 数据集中随机采样 100 条真实旋律。
- **随机噪声组**：由随机生成的 0 ~ 13 整数序列组成。
- **特殊序列组**：包含全 0 序列、简单音阶序列等人工构造的数据

4.1.2 适应度模型与评估策略

在确定初始种群后，我们利用第 3 节中训练好的 4 个神经网络模型作为评估器，并设置了以下三组对比实验展示 fitness model 的选取对于最终产生旋律的音乐特性之间的联系：

- **AI Mode**：仅使用神经网络的预测分数作为适应度函数 ($F = F_{AI}$)。
- **Hybrid Mode**：采用加权策略结合乐理规则与神经网络评分 ($F = 0.4 \cdot F_{rule} + 0.6 \cdot F_{AI}$)。
- **Rule Baseline**：仅使用乐理规则进行评估 ($F = F_{rule}$)，该组作为对照组。

我们对上述 种群-模型-架构的组合进行了迭代实验，并选取每一次生成种群中的最优个体进行可视化与分析。

4.2 fitness model 的选取对于最终产生旋律的音乐特性之间的联系

图 6 至图 11 展示了在不同种群-模型-架构下，Pop (蓝色)、ACG (橙色)、Classical (绿色)、Combined (红色) 以及 Rule Baseline (灰色) 五种模型生成的最终旋律可视化图。横坐标代表时间步 (0-63)，纵坐标代表相对音高 (1-13)。

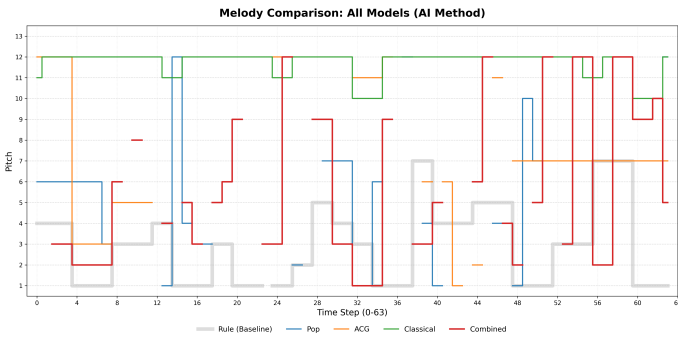


图 6: 将真实旋律片段作为初始种群 (AI 架构)

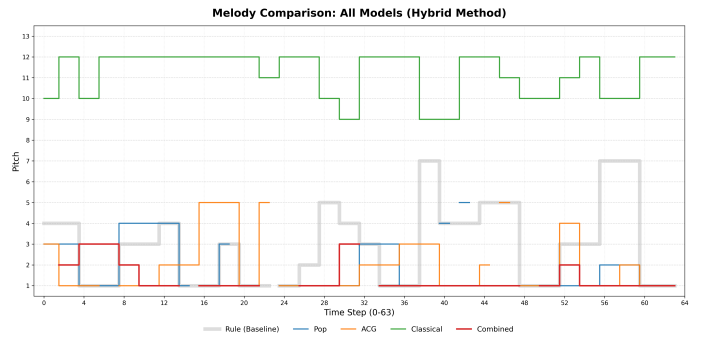


图 7: 将真实旋律片段作为初始种群 (Hybrid 架构)

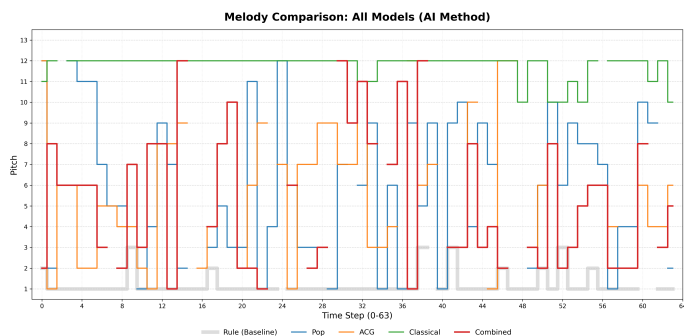


图 8: 随机生成音乐片段作为初始种群 (AI 架构)

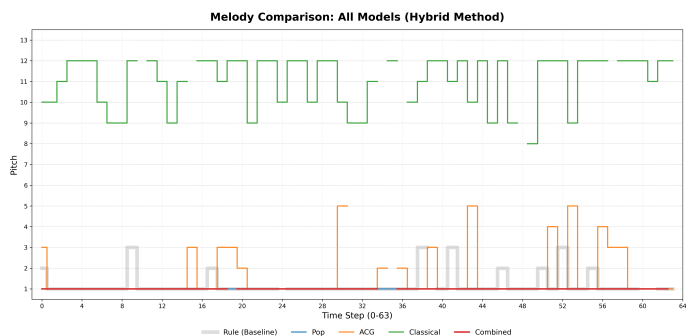


图 9: 随机生成音乐片段作为初始种群 (Hybrid 架构)

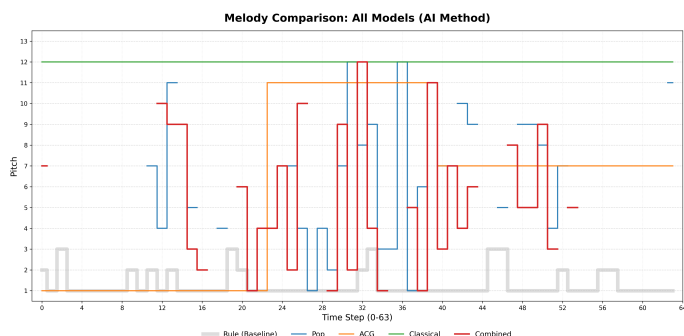


图 10: 特殊音乐片段作为初始种群 (AI 架构)

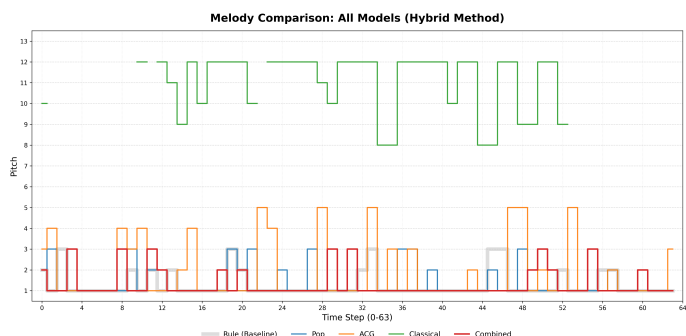


图 11: 特殊音乐片段作为初始种群 (Hybrid 架构)

4.3 可视化结果分析

通过横向 (AI / Hybrid 对比) 与纵向 (4 个神经网络模型对比) 的分析, 我们得出以下结论:

4.3.1 纯 AI 架构与 Hybrid 架构的对比

对比左列 (纯 AI 模式) 与右列 (混合模式) 的图像, 差异显著:

- **纯 AI 模式:** 生成的旋律 (图 6, 8, 10) 表现出明显的离散性和无序性。由于缺乏显式规则约束, 以神经网络为适应度函数时容易生成不具音乐特点的高分序列, 表现为音符在极高或极低音区的长时间滞留 (如 Classical 模型绿色线条), 或大幅度的非乐理性跳跃。
- **混合模式 (Hybrid):** 引入规则约束后 (图 7, 9, 11), 旋律的平滑度提升, 且有明显向 Rule baseline 收敛靠近的趋势。音程跨度回归到二度、三度为主的合理范围。由此可见, 规则充当了“正则化项”, 在纠正 AI 乐理错误的同时, 保留了神经网络对风格特征的引导作用。

4.3.2 不同神经网络架构的风格化影响

- **ClassicalResNet (绿色线条):** Classical 总体音级较高, 且表现出显著的相同音级连续重复, 这可能是因为 CNN 架构倾向于捕捉局部的重复模式, 导致生成的旋律经常出现小乐节的完全重复。

- **PopTransformer (蓝色线条)**: 生成的旋律线条最为平稳, 且重复周期较长。由于 Self-Attention 机制对长距离依赖的捕捉, Pop 模型倾向于生成跨小节的、结构规整的乐句, 这符合流行音乐中主歌/副歌的标准化结构特征。
- **ACGAttentionNet (橙色线条)**: 表现出更高的动态变化与节奏密度。在可视化中, ACG 模型的线条往往伴随着更频繁的起伏和短时值的切分, 这可以反映了 LSTM 对时序动态的敏感性以及 ACG 音乐追求情感爆发力的特点。
- **CombinedHybridNet (红色线条)**: 展现了最佳的平衡性与丰富度。Combined 模型生成的旋律在休止符运用和旋律灵活性方面做的最好, 听感最为自然。

4.3.3 初始种群分析

实验结果清晰地显示, 初始种群的质量直接决定了生成结果的优劣。在实际测试中, 基于随机噪声或特殊序列作为初始化种群的效果不佳, 生成结果在基于规则的适应度函数下得分显著偏低, 并产生了大量不合规的音乐片段。这体现出音乐序列并非杂乱无章的声音堆砌, 而是一种需要遵循特定的和声走向、节奏型态及旋律发展的具有高度组织性的艺术形式。

5 项目简介

本项目已在 GitHub 平台上开源<https://github.com/Stemiaya/MelodyGenome>
具体内容请详见 github 仓库的 README 文档。

5.1 使用说明

本系统提供了灵活的命令行接口, 支持用户从零开始训练模型或直接使用预训练模型进行生成。

5.1.1 模型训练

用户可以通过 `train.py` 训练评估模型。脚本会自动读取 `data/` 目录下的数据集, 并将最佳权重保存至 `checkpoints/` 文件夹。

```
# 训练特定风格的模型 (可选: acg, pop, classical, combined)
python train.py --target combined
```

5.1.2 旋律生成

训练完成后, 使用 `main.py` 启动遗传算法。用户可以自定义进化代数、种群大小以及适应度评估模式。

```
# 示例: 使用混合模式和组合模型进行生成
python main.py --seed 42 --gen 100 --pop_size 100 \
```

```
—mode hybrid —model combined \  
—output result.csv
```

生成的旋律将自动转换为 MIDI 文件并保存在 `output_midi` 文件夹中，方便用户直接试听。

6 总结与展望

6.1 总结

我们设计并实现了一个基于遗传算法与深度学习评估的机器作曲项目。我们构建了带有量化标签的专门数据集，并训练了多种神经网络模型作为遗传算法的评估器，有效解决了传统算法作曲中适应度函数设计单一的问题。

实验表明，引入逆行、倒影等音乐变换算子显著增加了种群的多样性；而采用“规则 + AI”的混合适应度函数，能够生成既符合乐理规范又具有一定风格特征的音乐片段。

6.2 不足与展望

由于 `musescore.com` 网站的爬取限制，我们获取的乐谱数量有限；且部分乐谱来自交响曲，不可避免高音声部会出现一些连续空拍或延长。尽管我们对数据进行了清洗，还是有一定改进空间。我们将乐音压缩到了一个八度之内，未来也可以探索音域的扩大、和弦伴奏的生成以及多声部协同进化算法的研究。

6.3 致谢

最后，由衷感谢王老师、毕老师和助教学长们一学期的付出。我们收获颇丰，对音乐、乐理有了更加深刻和不同视角的理解；我们在完成大作业的过程中收获了许多乐趣，也唤起了笔者十余年前练习电子琴的宝贵回忆。祝《音乐与数学》越来越好！

7 Appendix

由于篇幅限制，具体实验数据在 github 仓库的 `testData` 目录中呈现。

参考文献

- [1] Özcan, E., & Erçal, T. (2008). A Genetic Algorithm for Generating Improvised Music. In N. Monmarché, et al. (Eds.), *Artificial Evolution (EA 2007)*, Lecture Notes in Computer Science, vol 4926 (pp. 266-277). Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-79305-2_23